NETWORK TRAFFIC ANALYSIS REPORT

InternSpark Cybersecurity Internship
Task 3 — Wireshark Packet Analysis

Assessment type	Educational network traffic analysis
Analysis tool	Wireshark 4.6.9 with Npcap 1.88 support
Environment	Windows laptop; Wi-Fi interface; own-device normal network traffic
Evidence reviewed	Five supplied screenshots and a supplied PCAPNG capture
Report date	23 October 2026

Prepared for internship assessment. This report documents observed protocol behaviour and evidence; it does not assert that a vulnerability or attack was confirmed.

Table of Contents

1. Executive Summary

2. Introduction

3. Objectives

4. Scope and Environment

5. Tools and Technologies

6. Network Traffic Capture Methodology

7. DNS Traffic Analysis

8. TCP SYN Packet Analysis

9. TCP SYN-ACK Packet Analysis

10. TCP ACK Packet Analysis

11. TCP Retransmission Analysis

12. Summary of Findings

13. Recommendations and Best Practices

14. Limitations

15. Conclusion

16. Appendix: Display Filters Used

Evidence handling note: screenshots are reproduced from the supplied files. The supplied capture file was also reviewed structurally for packet-record totals and DNS-related traffic. No decrypted HTTPS payloads or sensitive packet payloads are reproduced.

1. Executive Summary

This assessment documents a practical Wireshark exercise performed on a Windows laptop using the Wi-Fi interface. The work focused on DNS queries and responses, TCP connection-establishment flags (SYN and SYN-ACK), ACK-only packets, and packets that Wireshark labelled as TCP retransmissions. The screenshots provide evidence of each requested display-filter view.

The supplied PCAPNG file was available for an additional structural review. It contains 42,480 Enhanced Packet Blocks (packet records). A basic packet-level review identified 222 UDP packets with source or destination port 53, consistent with DNS traffic, and visible DNS names included chatgpt.com and www.google.com. This lightweight review is not a substitute for a full Wireshark dissector-based re-analysis, and the screenshot counts are time-specific snapshots from a live capture that continued to grow.

No confirmed vulnerability, attack, or network fault is established by the supplied evidence. Retransmission labels warrant contextual review only; they may arise from packet loss, delayed acknowledgements, congestion, capture artefacts, or other conditions. The assessment is educational and limited to the observed capture window.

Overall result: the requested protocol behaviours were observed and documented. Further investigation would be needed to determine whether retransmissions were persistent, abnormal, or operationally significant.

2. Introduction

Wireshark is a packet-analysis tool that presents captured network frames in a structured, protocol-aware view. Display filters make it possible to focus on particular traffic classes without altering the original capture. This exercise used filters for DNS and selected TCP flags and analysis labels to understand common network protocol behaviour.

3. Objectives

The objectives were to: (1) capture live network traffic; (2) identify DNS queries and responses; (3) examine TCP SYN, SYN-ACK, and ACK packets; (4) observe packets labelled as TCP retransmissions; (5) understand basic packet structure and protocol behaviour; and (6) document the practical work with screenshots and a packet capture file.

4. Scope and Environment

The scope was limited to normal traffic visible on the assessor’s own Windows laptop Wi-Fi interface during a finite capture period. The evidence does not represent a full network-wide assessment, a controlled performance test, or a penetration test. The screenshots show different packet totals because capture continued while the filters were applied at different moments.

The report avoids reproducing private IP addresses and unrelated device identifiers from the screenshots. DNS names are included only as examples visible in the provided evidence.

5. Tools and Technologies

• Wireshark 4.6.9 — packet capture display, protocol inspection, and display filtering.
• Npcap 1.88 — packet capture support on Windows.
• Windows operating system and Wi-Fi capture interface.
• Supplied PCAPNG file — supplementary structural packet-record review.

6. Network Traffic Capture Methodology

1. Wireshark was opened and live traffic was captured from the Wi-Fi interface.
2. Display filters were entered to focus on DNS, TCP SYN, TCP SYN-ACK, ACK-only packets, and retransmission-analysis labels.
3. Representative filtered views were captured as screenshots for evidence.
4. The supplied PCAPNG file was reviewed for packet-record totals and DNS-related UDP traffic. The original capture was not modified for this report.
5. Observations were interpreted conservatively, distinguishing visible facts from possible explanations.

The display filters only change which packets are shown; they do not themselves prove why a packet occurred. Counts displayed in Wireshark are time-dependent when the capture is still running.

7. DNS Traffic Analysis

DNS (Domain Name System) allows a client to resolve a domain name into an IP address and to query other record types. The supplied DNS screenshot shows query/response traffic, with visible entries including www.google.com and chatgpt.com. The screenshot shows 5,138 total packets and 28 displayed DNS packets at that moment.

The PCAPNG structural review found 42,480 packet records in the supplied file and 222 UDP packets using source or destination port 53. This count is based on a lightweight packet-header review; it is separate from the screenshot's time-specific displayed count. The evidence confirms that DNS-related traffic was present, but it does not by itself indicate suspicious DNS behaviour.

Filter: dns

```
dns
```

Interpretation: this filter displays packets Wireshark decodes as DNS. Queries request records; responses may provide answers, aliases, or other DNS information. A DNS response can contain multiple records, and encrypted DNS protocols may not be visible under this simple filter.

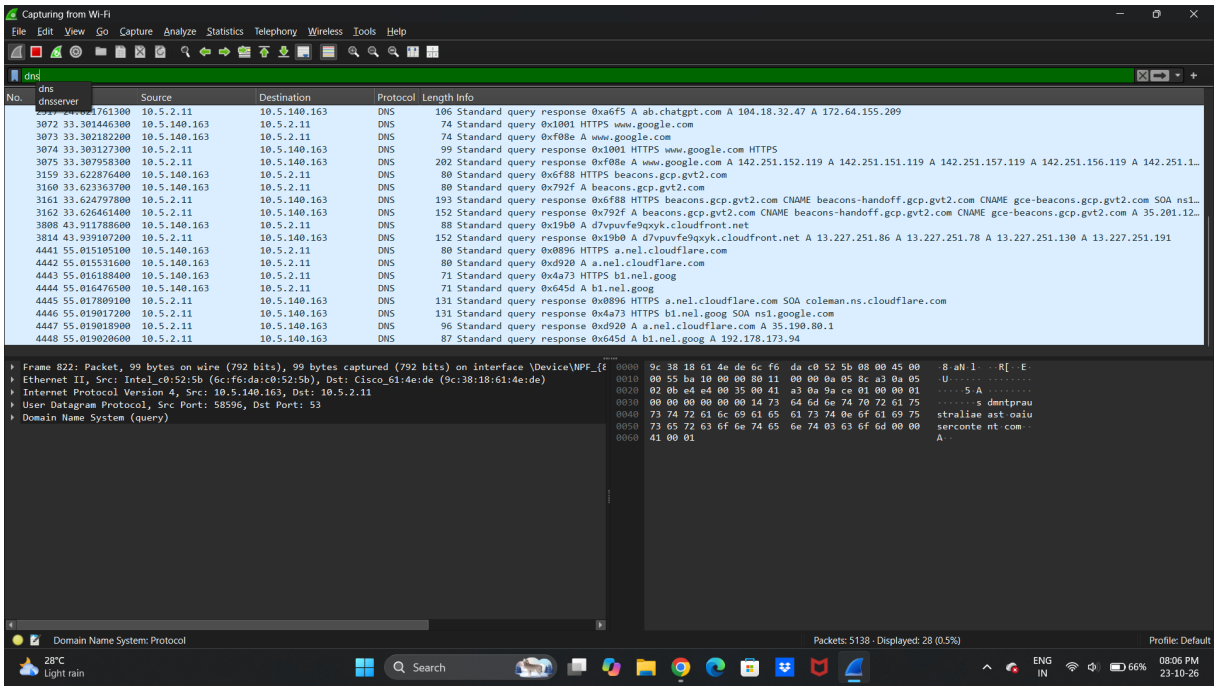


Figure 1. DNS display-filter view showing DNS queries and responses, including visible entries for www.google.com and chatgpt.com. The screenshot records 5,138 total packets and 28 displayed DNS packets at that point in the live capture.

8. TCP SYN Packet Analysis

A TCP SYN packet initiates a connection attempt. The filter tcp.flags.syn == 1 && tcp.flags.ack == 0 selects packets with the SYN flag set and ACK flag clear. The screenshot shows multiple initial SYN packets sent toward remote endpoints. Some entries appear more than once in the filtered view, but a filtered screenshot alone is insufficient to determine whether every repeated SYN reflects a retransmission, a new connection attempt, or another capture circumstance.

Filter

```
tcp.flags.syn == 1 && tcp.flags.ack == 0
```

Interpretation: SYN packets are normal TCP behaviour. An unanswered SYN or repeated attempt can be caused by a remote endpoint not responding, packet loss, filtering, or other network conditions. No malicious intent can be inferred from the flag alone.

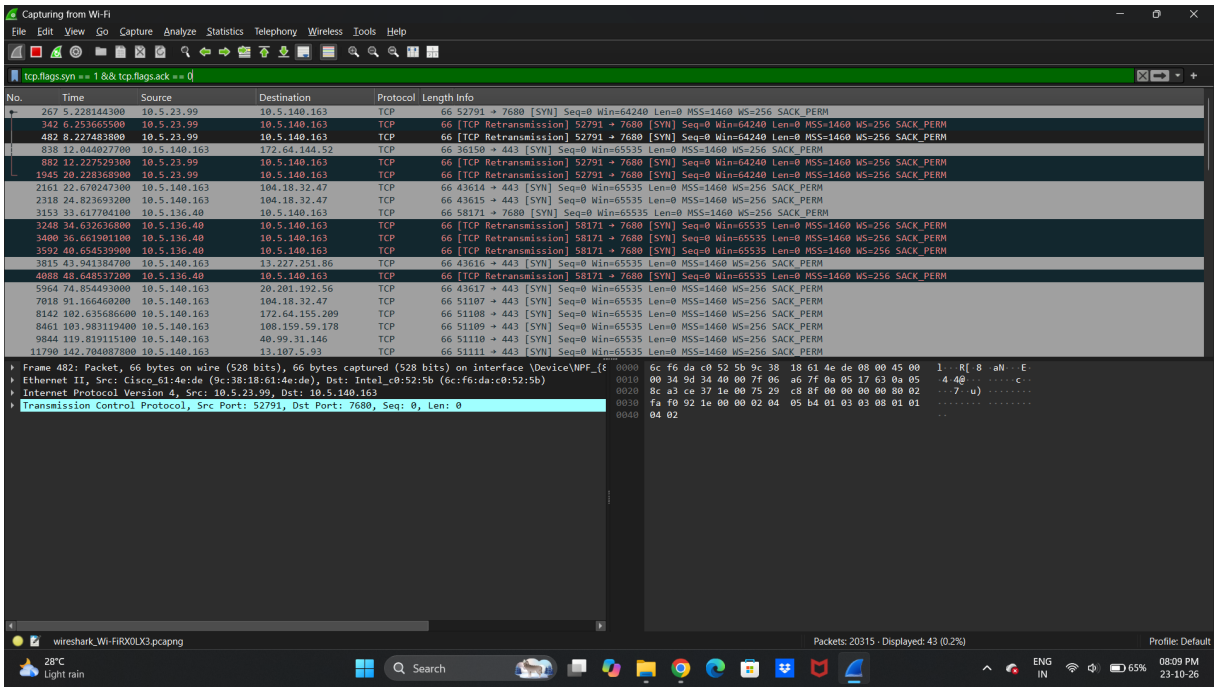


Figure 2. Initial TCP SYN packets filtered with tcp.flags.syn == 1 && tcp.flags.ack == 0. The view shows connection-initiation attempts; repeated SYNs can be consistent with retransmission or an unanswered connection attempt, but do not independently establish a fault or attack.

9. TCP SYN-ACK Packet Analysis

SYN-ACK is the response stage in the standard TCP three-way handshake. A server or peer acknowledging a SYN typically sets both SYN and ACK flags. The supplied screenshot displays packets matching the SYN-ACK filter and demonstrates that such responses were present in the capture.

Filter

```
tcp.flags.syn == 1 && tcp.flags.ack == 1
```

Interpretation: a SYN-ACK indicates a response consistent with TCP connection establishment. The screenshot does not prove that every handshake was captured completely; the corresponding final ACK may be outside the displayed subset or capture window.

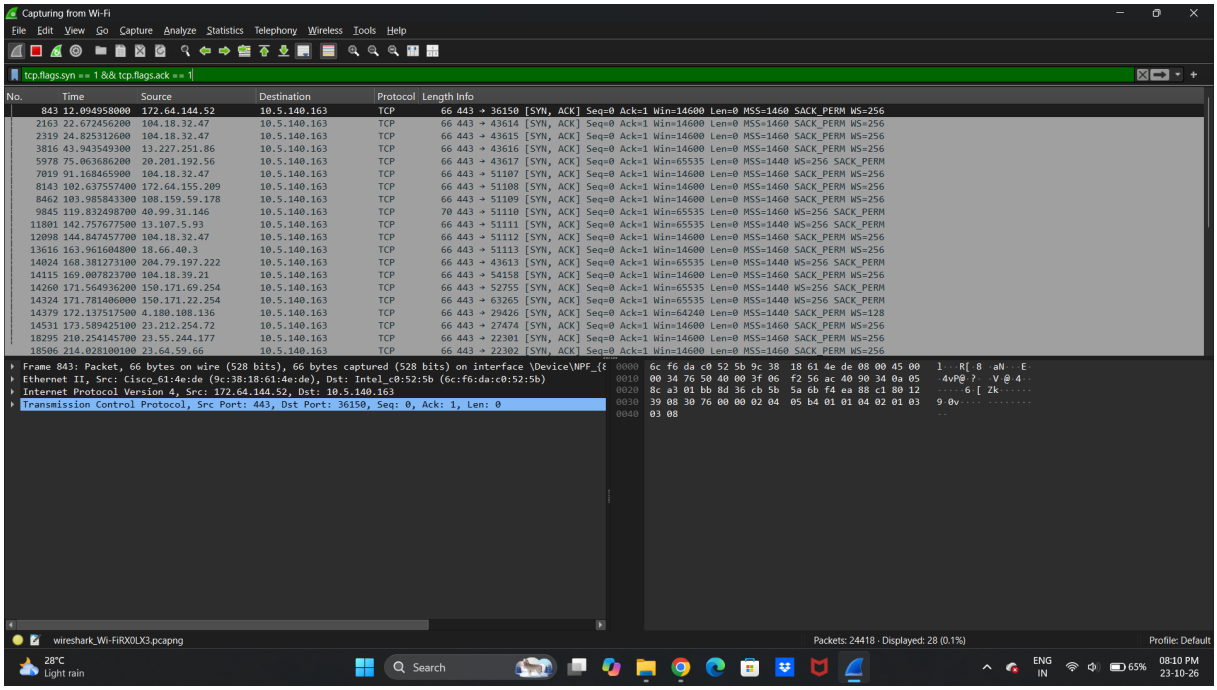


Figure 3. TCP SYN-ACK packets filtered with tcp.flags.syn == 1 && tcp.flags.ack == 1. These packets represent the server-side response stage of the TCP three-way handshake.

10. TCP ACK Packet Analysis

The ACK flag is used to acknowledge received TCP sequence space. The filter below selects TCP packets where SYN is clear, ACK is set, and TCP payload length is zero. The screenshot shows many zero-payload ACK packets, which are common during TCP communication.

Filter

```
tcp.flags.syn == 0 && tcp.flags.ack == 1 && tcp.len == 0
```

Important limitation: this filter is intentionally broad. It can include ordinary acknowledgements in established connections and is not a filter for only the third packet of the TCP three-way handshake. A packet's role must be interpreted with its surrounding flow and sequence/acknowledgement numbers.

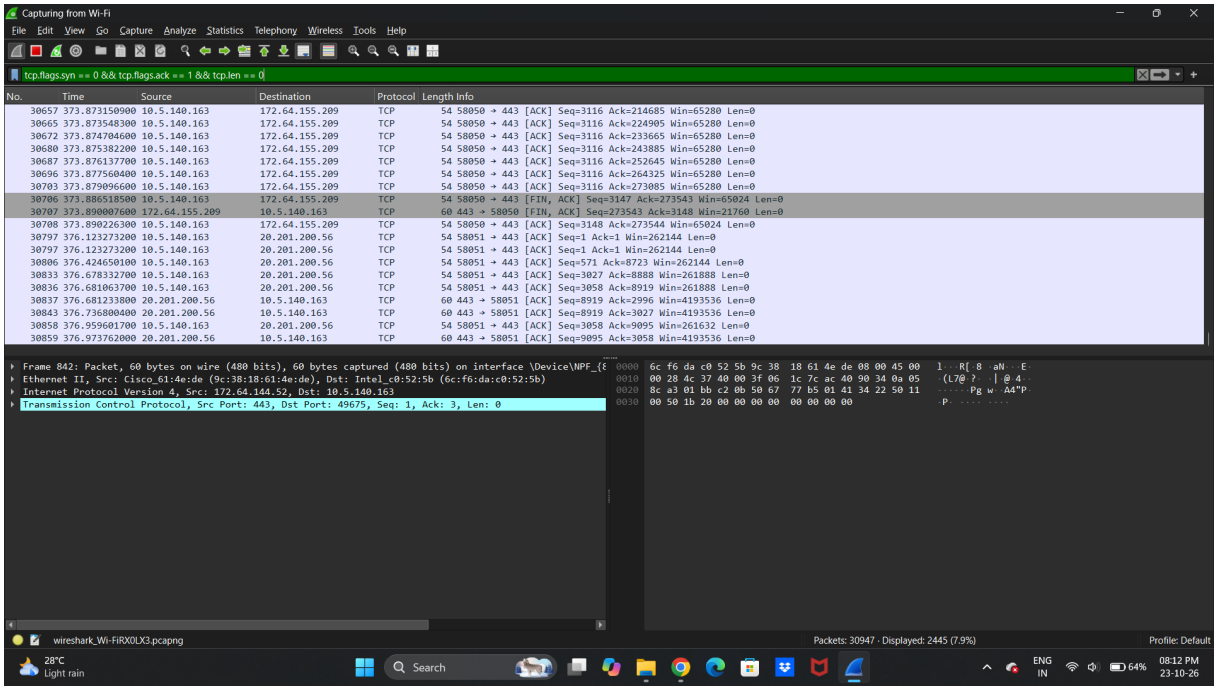


Figure 4. TCP ACK packets with no TCP payload filtered using tcp.flags.syn == 0 && tcp.flags.ack == 1 && tcp.len == 0. This broad filter includes ordinary acknowledgements as well as ACKs that may occur during connection setup or established data transfer.

11. TCP Retransmission Analysis

Wireshark's tcp.analysis.retransmission label identifies packets that its TCP analysis heuristics classify as retransmissions. The supplied screenshot shows 52 displayed packets at that moment, including repeated SYN attempts and data-bearing packets labelled as retransmissions. The exact count is tied to that screenshot and should not be treated as a fixed total for the entire capture.

Filter

```
tcp.analysis.retransmission
```

Possible explanations include packet loss, congestion, delayed or missing acknowledgements, a remote endpoint not responding in time, or capture-path effects. The label is an analytical clue, not a security verdict. To assess significance, inspect the full connection, compare sequence and acknowledgement numbers, check timing and direction, and repeat the capture under known conditions.

The evidence does not confirm a denial-of-service condition, malicious scanning, a network vulnerability, or a persistent connectivity fault.

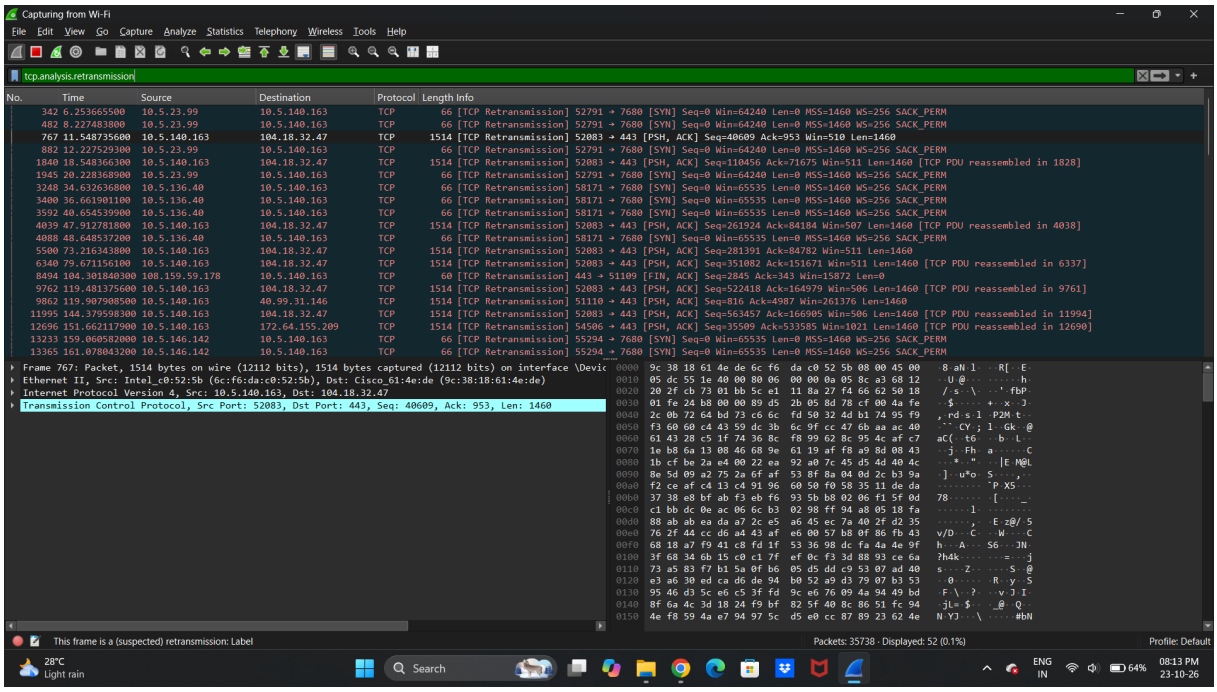


Figure 5. Wireshark retransmission-analysis view filtered with tcp.analysis.retransmission. The screenshot shows 52 displayed packets at that moment, including packets labelled as retransmissions. The labels are indicators for investigation, not proof of malicious activity or a confirmed network fault.

12. Summary of Findings

Observation	Evidence	Interpretation	Status
DNS query/response traffic visible	Figure 1; PCAPNG header review found 222 UDP packets involving port 53	DNS-related traffic was present; visible names include www.google.com and chatgpt.com. No suspicious DNS conclusion is supported.	Observed
TCP initial SYN packets	Figure 2; SYN filter view	Connection-initiation packets were present. Repetition alone does not establish an attack or fault.	Observed
TCP SYN-ACK packets	Figure 3; SYN-ACK filter view	Responses consistent with TCP connection establishment were present. Full handshakes are not claimed.	Observed
TCP ACK packets with zero payload	Figure 4; ACK-only filter view	Ordinary zero-payload acknowledgements were present; filter does not isolate only handshake-final ACKs.	Observed
TCP retransmission labels	Figure 5; 52 packets displayed at screenshot time	Retransmission candidates require flow-level/timing review; no attack or confirmed fault established.	Requires Further Investigation if persistent
Capture size and timing context	Supplied PCAPNG contains 42,480 packet records; screenshots show changing totals	Screenshots are snapshots from different moments; counts should not be compared as if simultaneous.	Observed

No severity rating is assigned because the evidence demonstrates protocol observations rather than a confirmed vulnerability.

13. Recommendations and Best Practices

• If retransmissions persist, take repeat captures over comparable periods and compare results before and after any network change.
• Follow an individual TCP conversation and compare timestamps, source/destination ports, sequence numbers, acknowledgement numbers, window values, and packet direction.
• Check whether the same connection shows repeated retransmissions, duplicate acknowledgements, out-of-order segments, or long response delays; avoid interpreting one packet in isolation.
• If symptoms persist, use appropriate troubleshooting tools such as ping, path testing, interface statistics, router/access-point logs, and application-side logs, as applicable and permitted.
• Keep PCAPNG files securely stored and share them only with trusted recipients. Captures may expose internal addresses, hostnames, device metadata, and potentially sensitive application information.
• Capture only traffic on devices and networks for which permission is available. Minimize capture duration and apply appropriate retention and deletion practices.
• Do not assume HTTPS payloads are readable. This assessment did not decrypt HTTPS payloads.

14. Limitations

• The capture represents a limited period of normal traffic on one laptop's Wi-Fi interface; it is not a complete view of all network activity.
• Screenshots were taken at different moments while packet capture continued, so displayed packet counts vary.
• A lightweight structural review of the supplied PCAPNG validated the packet-record total and identified UDP/53 traffic. It was not a full re-run through Wireshark's complete protocol-analysis engine.
• The evidence does not establish that every TCP handshake was fully captured.
• Retransmission analysis depends on capture position, timing, packet visibility, and Wireshark's heuristics. It does not alone establish a fault or attack.
• Encrypted HTTPS payloads were not decrypted or inspected.

15. Conclusion

The exercise successfully documented DNS activity, TCP SYN and SYN-ACK traffic, zero-payload ACK packets, and packets labelled by Wireshark as TCP retransmissions. The supplied capture file contains 42,480 packet records, and the structural review found DNS-related UDP traffic. The screenshots provide a traceable record of the practical work and the display filters used.

The main conclusion is observational rather than vulnerability-based: common network protocol behaviours were visible, and retransmission labels were present. Further connection-level analysis would be required to determine whether any retransmissions were unusual or operationally significant. No confirmed attack, vulnerability, or network fault is claimed.

16. Appendix: Display Filters Used

DNS traffic

```
dns
```

Displays packets decoded as DNS.

Initial TCP SYN

```
tcp.flags.syn == 1 && tcp.flags.ack == 0
```

Selects SYN packets with ACK clear.

TCP SYN-ACK

```
tcp.flags.syn == 1 && tcp.flags.ack == 1
```

Selects packets with both SYN and ACK set.

TCP ACK, no payload

```
tcp.flags.syn == 0 && tcp.flags.ack == 1 && tcp.len == 0
```

Selects ACK packets with SYN clear and zero TCP payload; not limited to handshake-final ACKs.

TCP retransmissions

```
tcp.analysis.retransmission
```

Displays packets Wireshark labels as TCP retransmissions.

Evidence inventory

Figure 1 — 01_dns_query_response.png
Figure 2 — 02_tcp_syn.png
Figure 3 — 03_tcp_syn_ack.png
Figure 4 — 04_tcp_ack.png
Figure 5 — 05_tcp_retransmission.png
Capture file — internspark_task3_capture.pcapng (supplied; approximately 17 MB).